

VERIQO

Evidence-Qualified Intelligence OS — Enterprise Architecture and Assurance Report

Repository: `veriqo` (Go 1.24.7) · **Branch:** `claude/veriqo-enterprise-architecture-7j56b9`
Report date: 5 September 2026

Status: NOT PRODUCTION READY. Twenty gates block release; thirteen of them cannot be satisfied by VERIQO at all. This report says so on its first page because a document that buries that on page forty is the failure mode the system it describes exists to prevent.

0. How to read this document

Every figure in this report is produced by code in the repository and can be regenerated:

```
./scripts/verify.sh      # 13 checks, and 12 explicitly-listed things it does NOT run
go run ./cmd/veriqoctl all # 8 reports, byte-identical between runs
go test ./...            # 699 tests across 50 packages
```

Nothing here is a summary that cannot be disagreed with. Where a number is an estimate it is labelled ESTIMATE at every point it appears, including inside the program that prints it. Where something has not been done, this report names it rather than omitting it.

1. What VERIQO is, and what it is not

Three systems already occupy adjacent ground:

System	World	Core question
Palantir	operational	<i>What is happening, and what should we do?</i>
Quantexa	contextual	<i>What is connected to what?</i>
Perplexity	information	<i>What does the corpus say?</i>

VERIQO occupies a fourth: the **proof and qualification world**. Its question is not *what is true* but ***what is this claim worth, and what would change it?***

That distinction is not marketing. It determines the whole architecture. A system that answers "what is happening" can afford to merge two records that probably refer to the same ship. A system that answers "what is this worth" cannot, because the merge itself is a claim, and an unqualified claim contaminates every conclusion downstream of it. So VERIQO has no merge

operation at all — resolution produces one of five outcomes and a merge is only one of them, reachable only with a reviewer.

VERIQO is neutral. It is not an accuser. It does not conclude fraud, liability or coverage; it qualifies the evidence on which somebody else concludes those things. Five acts are named in code as permanently forbidden to automation — `QUALIFY_FACT`, `APPROVE_MERGE`, `DECLARE_FRAUD`, `DECLARE_LIABILITY`, `DECLARE_COVERAGE` — and an artefact recording any of them fails validation rather than merely failing to be promoted.

2. The ten design laws, and where each one lives

The laws are not documentation. Each is enforced at a specific place, and each has a test that fails if the enforcement is removed.

#	Law	Enforced in	The failure it prevents
1	Evidence first	pkg/evidence/version	A claim with no evidence version cannot be constructed
2	Provenance first	pkg/provenance	SOURCE $\neq$ PRODUCER; ProducerID() walks to the first observer, so processing cannot launder origin
3	No silent merge	pkg/resolution	Five outcomes, not a boolean; contradiction is a VETO, not a negative weight
4	Every claim has a disproof path	pkg/claim	DisproofPath is a required field; a claim without one does not validate
5	Absence $\neq$ negative evidence	pkg/claim, pkg/uncertainty	"Missing" and "contradicting" are different states; NOT_ASSESSED ranks <i>below</i> NONE
6	Correlated sources do not count independently	pkg/qualification/independence	Six dimensions, three of them disqualifying; UNKNOWN never counts toward corroboration
7	AI cannot upgrade evidence	pkg/ai	QUALIFIED is unreachable by automation under <i>any</i> policy — AutomatedPolicy.Validate refuses one that permits it
8	Replayability is mandatory	pkg/replay, pkg/canonical/jcs	No deterministic path reads the wall clock; a replay that re-executed nothing reports that it establishes nothing
9	Immutable decision lineage	pkg/ledger, pkg/evidence/version	Version 1 is the raw acquisition and is never superseded; the ledger's hash chain covers height and predecessor
10	Security is part of semantics	pkg/tenant, pkg/governance/classification	Classification is a two-axis lattice with a <i>partial</i> order; tenancy is a cryptographic anchor, not a WHERE clause

2.1 Law 7 in detail, because it is the one people expect to be soft

There are three independent barriers, and the design intent is that all three must be removed to reach a qualified fact by automation:

1. Act.Permitted() — an artefact recording a forbidden act is malformed and cannot exist.
2. Promote() — one level at a time, never by the producer, and an automated promoter needs a named policy for a declared risk class.

3. `AutomatedPolicy.Validate()` — a policy whose `MaxLevel` is `Qualified` is itself refused. There is no policy that can be written to permit it.

`test/adversarial` attempts the climb one automated rung at a time and asserts the ceiling holds.

3. Shape of the system

50 packages, 119 Go files, ~37,700 lines, 54 test files, **699 tests**.

```
pkg/canonical/jcs      RFC 8785 canonicalisation — everything hashes through here
pkg/contract           four-valued Outcome, VersionSet, deterministic ID, Clock
pkg/identity           principals: HUMAN SERVICE AGENT DEVICE CONNECTOR SOURCE
pkg/tenant             cryptographic tenant anchor, per-surface key derivation
pkg/governance/...     two-axis classification lattice
pkg/authority          9 roles x 7 capabilities, separation of duties
pkg/policy             deny-overrides ABAC with an unoverridable core
pkg/rights             six licence questions, combined by intersection
pkg/ledger             durable WAL, hash chain, signed checkpoints
pkg/audit              Guard(): work runs only inside a recorded operation
-----
pkg/evidence/...       versions, quality, redaction worker + corpus
pkg/provenance/...     hop path, producer resolution, temporal standing
pkg/custody            chain of custody with permanent, recorded breaks
pkg/entity            typed identifiers with strength and reassignability
pkg/resolution         10 signals, 5 outcomes, contradiction as veto
pkg/ontology          16 object types, 22 relationships, 6 domain views
pkg/graph             one store, domain projections, weakest-link paths
-----
pkg/claim             disproof path mandatory, contradictions demote
pkg/reverseproof       no CONFIRMED verdict exists
pkg/hypothesis        ACH scoring over a set, not a winner
pkg/quantum           refuses to subtract incomparable measurements
pkg/uncertainty        9 dimensions and deliberately NO aggregate
pkg/trust             6 kinds, with no path from source to conclusion
-----
pkg/findings          one mint authority, guarded by an unexported witness
pkg/casefile          resolution requires a gate record, not just order
pkg/passport          limitations inside the signed payload
pkg/replay            DETERMINISTIC vs RECORDED steps, distinguished
-----
pkg/ai                the five-level ladder
pkg/agents            the tool firewall
pkg/modelregistry      DEVELOPMENT -> VALIDATION -> QUALIFIED -> PRODUCTION
pkg/connectors        no vendor names; producer and licence required
pkg/security/kms       SoftwareRoot is a TEST DOUBLE, refused in production
pkg/resilience       breaker, bucket, bulkhead, idempotency, backpressure
pkg/api              34 endpoints; a missing guarantee fails registration
-----
pkg/assurance/...     failure classes, self-doubt register
pkg/gates            20 permanent production gates
pkg/scorecard         9 dimensions, no aggregate score
pkg/qualification/... assurance ladder, source independence
-----
test/architecture     the rules the source must obey
test/integration      the canonical pipeline, end to end
test/adversarial      43 tests that assume an attacker
test/mutation         the tests that must fail when an invariant is broken
```

3.1 The one dependency everything else rests on

`pkg/canonical/jcs` implements RFC 8785 (JSON Canonicalization Scheme). If it is wrong, every digest, every ledger record, every passport and every replay comparison in the system is wrong, and wrong *silently*.

Three properties in it are easy to get wrong and are pinned by tests:

- **ECMAScript number serialisation.** `1e21` renders `1e+21`, not `1000000000000000000000`. A naive `strconv.FormatFloat(f, 'g', -1, 64)` diverges at boundaries reachable from ordinary data — a large monetary amount in minor units, a coordinate delta.
- **UTF-16 code-unit key ordering.** U+1F600 sorts *before* U+FFFD in UTF-16 and *after* it in UTF-8 byte order. Only one of those is RFC 8785, and taking the wrong one produces a system that works perfectly and silently stops interoperating with every other implementation.
- **Invalid UTF-8 is refused, not repaired.** `encoding/json` silently replaces invalid UTF-8 with U+FFFD, and that repair happens *before* anything downstream can observe it — so the digest would cover bytes nobody supplied. A reflective pre-pass rejects the input instead.

4. The canonical pipeline

Twenty-one stages, each one's output being the next one's required input. `test/integration` drives all of them on a real cargo-discrepancy case and asserts the composition holds.

```
SOURCE -> ACQUISITION -> RAW EVIDENCE + HASH + CUSTODY -> NORMALIZATION
-> EVIDENCE QUALITY -> ENTITY RESOLUTION -> ONTOLOGY / GRAPH -> CLAIM
-> REVERSE PROOF -> HYPOTHESIS SET -> COUNTERFACTUAL -> CONTRADICTION MATRIX
-> SOURCE INDEPENDENCE -> TRUST + UNCERTAINTY -> SELF-DOUBT -> QUALIFICATION
-> HUMAN REVIEW -> FINDING -> PASSPORT -> REPLAY / AUDIT
```

The worked case: 60,000.000 MT loaded, 58,200.000 MT discharged, a 0.5% contractual tolerance on the *contract* quantity (300 MT), leaving 1,500 MT in excess of tolerance and, at the contract price, USD 930,000.

Four things about that number are what the system is for:

1. It is **basis-checked**. `pkg/quantum` refuses to subtract two measurements taken on incomparable bases — different method, temperature, standard, or in-air versus in-vacuum. "Not stated" counts as a difference, not as agreement.
2. It carries an **alternative construction**. Whether the tolerance is deducted from the claim or merely triggers it is a question of contract construction; the two readings differ by 300 MT, and the alternative travels with the figure rather than being dropped.
3. Its **two surveys are two sources**. The loading and discharge surveys have separate producers, separate provenance records and separate root evidence versions. If they had shared a producer they would count as one observation, and the system says so.

4. The **weak dimensions reach the finding**. The confidence vector's LOW completeness dimension arrives in the finding's limitations without anybody copying it there, and from there into the signed passport payload.

5. The failure-class register

Ten failure classes, each carried through eight stages: FINDING → ROOT CAUSE → FAILURE CLASS → INVARIANT → POSITIVE TEST → NEGATIVE TEST → MUTATION TEST → REGRESSION TEST.

10 closed findings across 10 failure classes; 40 tests cited, and all 40 exist. A self-referential check in `verify.sh` fails the build if any cited test name is absent — which is how three stale citations were found in this repository and corrected.

Class	What went wrong
FC-001	UNQUALIFIED_ASSERTION — a finding could be constructed without going through the mint
FC-002	UNVERIFIED_DERIVATIVE — verification searched raw bytes rather than the inspectable view
FC-003	FALSE_CORROBORATION — unassessed source pairs counted toward independence
FC-004	STALE_CITATION — a test cited by an invariant no longer existed
FC-005	SILENT_COERCION — a non-finite number canonicalised to <code>null</code>
FC-006	SELF_QUALIFICATION — a proposer could approve their own work
FC-007	UNGATED_RESOLUTION — a perfectly ordered case with no gate record passed
FC-008	LOST_LIMITATION — a limitation dropped between finding and passport
FC-009	OFFSETTING_ATTRIBUTES — a strong quality attribute stood in for a missing one
FC-010	IRREVERSIBILITY_OVERCLAIM — hiding content and removing it were indistinguishable to the verifier

6. Redaction coverage — Article 18

This is the section where honest reporting matters most, because the temptation to publish a single reassuring percentage is strongest.

23 structural variants: 17 accepted, 6 refused by design, 0 failed.

- **Structural coverage: 74%.** This is the share of *variants* the workers can redact. It is **not** a pass rate. Every refusal is safe, and none of them is a capability failure.
- **Real-world weighted coverage: 88% (ESTIMATE).** The same result weighted by how common each structure is in real documents. This is the more meaningful figure: 100% coverage of rare structures would be worse than 80% of common ones.

Three of the six refused variants are COMMON in real documents — `PDF-ENCRYPTED`, `PDF-INCREMENTAL`, `PPTX-EMBEDDED-IMAGE`. A real-world population would land there in bulk. The 88% figure would fall.

The prevalence weights are stated estimates, not measurements. VERIQO has never run this pipeline over a real document population. They are published so the weighted figure can be recomputed against better numbers, and so a reader can disagree with the estimate rather than with a hidden assumption.

Corpus qualification level: L2_FIXTURE_CORRECTNESS. Every fixture was built by VERIQO. No document in this run came from outside. No party outside VERIQO has attempted to recover redacted content from a derivative. *Surviving one's own corpus is the weakest form of survival available.*

6.1 The rule the worker turns on

Anything the worker cannot decode is **refused**, never reported clean. A document nobody could read is a document nobody can certify as redacted. Encrypted PDFs, LZW streams, incrementally updated files, malformed structures and embedded binary objects are all refused with a named structure and a stated reason — an unexplained refusal is indistinguishable from a bug.

7. The adversarial suite

`test/adversarial` contains **43 tests that assume an attacker**. Each asserts the *structural* reason an attack cannot work, not that it happened not to.

Area	Attacks
Prompt injection	grant widening mid-run; out-of-scope case ids; purpose switching; reaching export and approve; budget exhaustion; unconstrained scope read as unrestricted; firewall overriding policy; expired agent
Tenancy	key and namespace collision across tenants; a rebuilt tenant inheriting old keys; the field-concatenation attack ("ab"+"c" vs "a"+"bc"); Guard against nine wrong tenants including empty and zero-valued scope
Classification	derivative marked below its sources; a caveat silently dropped; clearance at a higher level without the compartment
Tampering	an edited ledger record; a log truncated from the front; a checkpoint presented for a different chain; an unanchored checkpoint; substituted content under an honest-looking custody link; appending to a sealed chain; forged signature bytes
Documents	undecodable structures; a decompression bomb; truncated, empty and mistyped containers; an empty redaction term; case-variant evasion; a manifest with no stated limits
Laundering	a model promoting its own output; the ladder jumped; QUALIFIED reached by climbing; a forbidden act recorded; a hand-written history that does not reach its level; self-approval; an internal or self-attesting "independent" assessor; one registry bought from two resellers; an unassessed pair counted as corroboration

7.1 What it found

The suite produced counterexamples on its first run. Three real defects, all now fixed:

1. The ledger was not tamper-evident in the middle of a log. A checksum failure *anywhere* was treated as a torn tail. Editing record 2 of 4 therefore silently discarded records 2, 3 and 4 and reopened the chain at height 2 — with no error. Tamper-and-truncate. *Fixed:* a torn write is by definition the last thing in a file, so damage is now accepted as a tail only when what follows it is absent or zero-filled. Anything else is a chain break and the ledger refuses to open.

2. The same path let a log truncated from the front open as an empty chain. Closed by the same fix.

3. The redaction worker silently truncated oversized parts and released them as verified. Every decompressed read used `io.LimitReader`, which *succeeds* and returns a prefix. A part inflating to 256 MiB was truncated to 64 MiB, redacted, released and marked `Verified` — with 192 MiB absent from the derivative and never searched for terms. Truncating a document and calling it clean is the worst outcome that package has. *Fixed:* `readBounded` refuses at the ceiling instead of truncating, with a declared-size pre-check in front of it.

7.2 What that means, stated carefully

Two of the three defects were in exactly the controls the system's own marketing would rest on: the immutable ledger and the verified derivative. They were found by the party that wrote them. That is the strongest available evidence that the disproof paths are real — and it is also

a reminder that these attacks were designed by somebody who knew where the code looked. **Gates G15, G16, G17 and G18 exist because that is not the same as being attacked.**

8. The self-doubt register

Fourteen claims, each with a proof path *and* a disproof path — not a negative test, but an attempt to produce a counterexample.

Claims record three outcomes: **ESTABLISHED** (proof succeeded, disproof found nothing), **REFUTED** (a counterexample exists; the claim is demoted to IMPLEMENTED and must name it), and **UNSETTLED** (the proof succeeded and the disproof was not or could not be run — *not* established).

The register now also records **closed counterexamples**: a defect the disproof path did produce and that has since been fixed, with the fix cited. This distinction matters. A claim that has never yielded to attack may mean the control is sound or may mean the attack was weak. A claim whose attack once succeeded is a claim whose attack is *known to be capable of succeeding*. Deleting that history when the fix lands throws away the only evidence that the disproof path is real.

A test asserts the register as a whole: if not one disproof path had ever produced a counterexample, either the system is perfect or the attacks are not real, and the second is far more likely.

Every claim in this register was attacked by VERIQO and by nobody else. The register says so on every claim rather than once in a preface.

Deliberately UNSETTLED, among others:

- **CLAIM-REDACTION-IRREVERSIBLE.** Absence in twelve encodings is not the same claim as irrecoverability, and nothing in this repository attempts recovery.
 - **CLAIM-INJECTION-STRUCTURALLY-REFUSED.** Ten adversarial tests attack it and none has produced a counterexample — which is precisely the weaker position.
-

9. The twenty gates

Twenty permanent production gates. **All twenty are blocking. Thirteen cannot be satisfied by VERIQO alone**, because they require a party that is not VERIQO:

Gate	Requires
G1	An HSM vendor or cloud KMS provider, with an attestation
G2	A commercial data provider
G4	An accredited penetration testing firm
G7	A SPIRE deployment operated separately from the workloads
G8	A vulnerability database provider
G9	A customer or industry body supplying a document corpus
G10	An evidence provider willing to confirm a content hash
G15	A red team that is not the team that built the isolation
G16	A red team with AI-agent experience
G17	A red team with AI-agent experience
G18	A red team
G19	A signing authority and an attestation service
G20	An evaluation set VERIQO did not construct

The remaining seven (G3, G5, G6, G11, G12, G13, G14) are within VERIQO's power and are unsatisfied because the environment for them — multi-region, multi-host, a timed disaster recovery, a 72-hour soak — does not exist here.

VERIQO deliberately **does not implement** the `ledger.Anchor` interface. An anchor whose implementation VERIQO controls proves that VERIQO agrees with itself.

10. The enterprise scorecard

Nine dimensions, rated GREEN / YELLOW / RED. **There is no aggregate score, deliberately** — a single figure would let a strong dimension carry a weak one, and the weak one is what a customer needs to see.

Dimension	Rating
EVIDENCE_INTEGRITY	YELLOW
ENTITY_INTEGRITY	YELLOW
REASONING_INTEGRITY	YELLOW
SECURITY	RED
OPERATIONAL_RELIABILITY	RED
DATA_RIGHTS	YELLOW
AI_GOVERNANCE	YELLOW
REPLAYABILITY	YELLOW
EXTERNAL_VALIDATION	RED

Nothing is GREEN. The release rule is: no RED, *and* every mandatory gate satisfied. Neither holds.

RELEASE NOT PERMITTED — 9 reasons. Three RED dimensions and six mandatory gates blocked on an external party (G1, G2, G7, G8, G19, G20).

EXTERNAL_VALIDATION's RED is the one that bounds every other rating: *no party outside VERIQO has examined, attacked, validated or corroborated any part of this system*. Every other dimension's YELLOW sits underneath it.

11. What is deliberately absent

A list of things a reader might expect and will not find, with the reason:

- **An aggregate confidence score.** `pkg/uncertainty` has nine dimensions and no `Overall()`. `Weakest()` returns a *dimension*, not a number.
- **A CONFIRMED verdict.** `pkg/reverseproof` has no such outcome. Evidence can fail to disconfirm; it cannot confirm.
- **A merge operation.** `pkg/resolution` produces five outcomes and a merge needs a reviewer.
- **A trust path from source to conclusion.** `pkg/trust` has six kinds of trust and no propagation from a source's or a model's trust to a conclusion's.
- **Vendor names in the connector layer.** A connector requires a `ProducerID` and a licence; discovery and acquisition are separately licensed.
- **A production KMS.** `kms.SoftwareRoot` is a test double and refuses to run in production mode. G1 is the gate.
- **An anchor implementation.** See §9.

- `time.Now()` **on any deterministic path**. Clocks are injected. An architecture test enforces it.
-

12. Verification

`./scripts/verify.sh` runs **13 checks, all passing**, including four honesty checks that fail the build if the system starts overstating itself:

- no gate is satisfied (the moment one is, this check must be updated deliberately)
- coverage figures carry the word ESTIMATE
- the self-doubt register states who attacked each claim
- every test cited by a failure-class invariant exists

It then prints **12 things it explicitly does NOT run**, each tied to the gate that would cover it: `golangci-lint`, `govulncheck`, `gosec`, SBOM generation, SPIRE attestation, OPA bundle validation, the 72-hour soak, multi-host qualification, multi-region failover, disaster recovery timing, an independent penetration test, and an external document corpus.

Passing the script means the code builds, the tests hold, and the system still refuses to overstate itself. **It does not mean VERIQO is production ready.**

13. Conclusion

The engineering is complete in the sense the specification asks for: fifty packages, the canonical pipeline composing end to end, ten failure classes closed with forty cited tests that all exist, an adversarial suite that has already found and closed three real defects in the system's own core controls, and a reporting surface that refuses to publish a number without publishing what it rests on.

The engineering is *not* complete in the sense that matters commercially, and the system says so itself, in code, on every run: nothing is GREEN, twenty gates block, thirteen of them need somebody who is not VERIQO, and release is refused for nine stated reasons.

That gap is not a defect in this report. It is the report's principal finding — and a platform that reports it accurately about itself is the only kind that can be trusted to report it about anything else.

Generated from the repository. Every figure regenerable with `go run ./cmd/veriqctl all`.